TOOLKIT : BACKEND ENGINEERING TOOLS & INFRASTRUCTURE HANDBOOK

Git Complete Handbook

Developer Foundations, Collaboration, Recovery & Production Engineering Protocols

---

CURRICULUM PHASE

Phase I — Developer Foundations (Tool 01 / 13)

SCOPE & COVERAGE

Chunks 1 & 2 Complete (Full 32-Chunk Curriculum)

TARGET AUDIENCE

Aspiring Backend Engineers & SDE-1 Candidates

DELIVERABLE SPECIFICATION

BEH_GIT_COMPLETE_HANDBOOK.pdf

METHODOLOGY

3-Layer Model: Fundamentals, Workflows, Production

QUALITY STANDARD

Authoritative, Zero-Omission Reference

Notice: This handbook is an authoritative educational deliverable from the *ToolKit* curriculum. It integrates complete conceptual foundations, step-by-step terminal workflows, practical command usage, production incident diagnostics, 30 rigorous interview questions with model answers, hands-on laboratory exercises, and exhaustive operational readiness checklists.

Table of Contents

PART I: GIT FUNDAMENTALS & ESSENTIAL WORKFLOWS (Chunk 1 of 2)

1. What Is Git? (Mental Models, Git vs. GitHub) Section 1
2. The Three Main Areas of Git (Working Tree, Staging Area, Local Repository) Section 2
3. Git's Core Vocabulary (Repository, Commit, Branch, HEAD, Remote, Merge) Section 3
4. Configure Git (Global vs. Local Identity Configuration) Section 4
5. Create or Obtain a Repository (git init vs. git clone) Section 5
6. Your First Complete Git Workflow (Inspect, Edit, Review, Stage, Commit) Section 6
7. Essential Git Commands (Quick Reference Table) Section 7
8. Branching for Backend Development (Feature Isolation) Section 8
9. Understand fetch, pull, and push (Remote Synchronization Protocols) Section 9
10. Merge Conflicts (Detection, Conflict Markers & Step-by-Step Resolution) Section 10
11. restore, revert, and reset (Safe Undos & Destructive Modes) Section 11
12. Temporarily Store Work with git stash (Context Switching) Section 12
13. .gitignore and Secret Protection (Credential Leakage Prevention) Section 13
14. Pull Requests and Code Review (Collaboration Guidelines) Section 14
15. Common Git Mistakes (Root Causes & Preventions) Section 15
16. Practical Lab — Create a Clean Git Repository (Hands-On Lab Tasks A-H) Section 16
17. Git Interview Questions and Answers (Q1 to Q15 with Detailed Rationale) Section 17
18. Mastery Checklist (Phase I Core Readiness Verification) Section 18

PART II: COLLABORATION, RECOVERY & PRODUCTION ENGINEERING (Chunk 2 of 2)

1. The Mission: From Using Git to Engineering with Git (The 3-Layer Architecture) Section 1

LAYER 1 — FUNDAMENTALS & MENTAL MODELS

2. Git Collaboration: Understand the Repository You're Working In (Remotes, Upstreams) Section 2
3. Understand History Before Manipulating It (Topology, Ancestry, Hashes) Section 3
4. Merge, Rebase, and Cherry-Pick: Three Different Decisions (Trade-offs & Decision Table) Section 4

LAYER 2 — PRACTICAL ENGINEERING & WORKFLOWS

5. The Professional Feature-Branch Workflow (8-Step Production Protocol) Section 5
6. Scenario: Your Push Is Rejected (Non-Fast-Forward Diagnosis & Remediation) Section 6
7. Scenario: You Committed to the Wrong Branch (Unpublished vs. Shared History) Section 7
8. Scenario: A Commit Introduced a Regression (Revision Inspection Techniques) Section 8
9. Find the Problematic Commit with git bisect (Binary Search for Bugs) Section 9
10. Recovery: You Cannot Find a Commit (reflog Investigation & Recovery Branches) Section 10
11. Recovery: Choosing Between Reset and Revert (Decision Matrix) Section 11

LAYER 3 — PRODUCTION ENGINEERING & INTERVIEW MASTERY

12. Production Incident: A Bad Configuration Reaches Production (Incident Simulation)	Section 12
13. Git Safety Rules for Production Teams (Six Golden Operational Rules)	Section 13
14. Practical Engineering Lab (Advanced Hands-On Labs A through E)	Section 14
15. Advanced Interview Questions (15 Architectural & Production Scenarios Q1-Q15)	Section 15
16. Git Mastery Assessment (Full 3-Layer Competency Matrix)	Section 16
17. Git Handbook Completion Protocol & Next Curriculum Milestones	Section 17

PART I: GIT FUNDAMENTALS & ESSENTIAL WORKFLOWS

Phase I — Developer Foundations | Chunk 1 of 2

Git Fundamentals

Backend Engineering Tools & Infrastructure Handbook

Phase I — Developer Foundations | Chunk 1 of 2

1. What Is Git?

Git is a distributed version control system that tracks changes to files over time.

Imagine you're building a Spring Boot backend. You add authentication, modify the database layer, and change your API validation logic. Suddenly, the application stops working.

Without version control, you might have to manually reconstruct what changed.

With Git, you can inspect previous changes, identify the problematic commit, collaborate safely, and restore a working version.

Git helps you answer four essential questions:

1. What changed?
2. When did it change?
3. Who made the change?
4. How can I recover or build upon an earlier version?

Git versus GitHub

These are different things.

Git	GitHub
Version control software	Online platform for hosting Git repositories
Runs locally	Provides remote repository hosting
Tracks changes and commits	Enables pull requests, code reviews, and collaboration
Works without an internet connection	Usually accessed over a network

Mental model: Git is the version-control engine; GitHub is one place where your Git repository can live.

Other hosting platforms include GitLab and Bitbucket.

2. The Three Main Areas of Git

Understanding these three areas makes most everyday Git commands easier to understand.

Imagine modifying a Java file called `UserService.java`.

Your changes move through three conceptual areas:

```
Working Directory
       |
       | git add
       v
Staging Area
       |
       | git commit
       v
Local Repository
```

2.1 Working Directory

The working directory contains the files you currently see and edit.

For example:

```
public class UserService {
    public String getStatus() {
        return "ACTIVE";
    }
}
```

You modify the method to return a different value.

The change exists in your working directory, but you have not necessarily staged or committed it.

2.2 Staging Area

The staging area contains the changes you intend to include in your next commit.

```
git add UserService.java
```

This stages the current changes to that file.

Why have a staging area?

Because you may modify five files but want to commit only two of them.

Staging allows you to construct a deliberate, logically grouped commit.

2.3 Local Repository

When you commit staged changes, Git records a new snapshot in the local repository.

```
git commit -m "Add user status handling"
```

A commit records information such as:

- The tracked project snapshot.
- A commit identifier.
- The author and timestamp.
- A commit message.
- References to parent commits.

Git commits are local by default. They are not automatically uploaded to GitHub.

The Critical Distinction

- `git add` stages changes.
- `git commit` records staged changes locally.
- `git push` sends commits to a remote repository.

These commands perform different jobs.

---

3. Git's Core Vocabulary

Learn these concepts before moving to advanced workflows.

Term	Meaning
Repository	A project and its Git history
Commit	A recorded snapshot of project state
Branch	A movable reference to a line of development
HEAD	Usually identifies the currently checked-out commit or branch
Remote	A reference to another repository, commonly hosted online
Staging area	The changes selected for the next commit
Merge	Combines histories from different development lines
Merge conflict	A situation where Git cannot automatically reconcile competing changes
.gitignore	Specifies untracked files Git should normally ignore
Working tree	The checked-out files you work with

What is a branch?

A branch lets you develop a feature independently of another line of development.

For example:

```
main
 |
 A
 |
 B
 |
 C
```

Suppose you create a feature branch from commit B:

```
A --- B --- C       main
       \
        D --- E     feature/auth
```

You can develop authentication on **`feature/auth`** without immediately incorporating it into **`main`**.

A branch is not a complete independent copy of the repository's history. It is a reference to a commit, and new commits advance that reference.

---

4. Configure Git

Before making commits, configure the identity recorded in your commits.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Replace the placeholders with your preferred commit identity.

Check your configuration:

```
git config --global --list
```

To configure a different identity for just one repository, run these commands from that repository and omit **`--global`**:

```
git config user.name "Your Name"
git config user.email "you@example.com"
```

What does `--global` mean?

- With **`--global`**, the setting applies to your user account's Git configuration.
- Without **`--global`**, the setting applies to the current repository.

Git commit identity is metadata. It does not, by itself, authenticate you to GitHub.

---

5. Create or Obtain a Repository

There are two common ways to begin.

5.1 Initialize a Local Repository

If you already have a project directory and want Git to track it:

```
git init
```

Run this from the intended project directory.

Git creates the repository metadata needed to track the project's history.

Check the current state:

```
git status
```

If appropriate, establish main as the initial branch:

```
git branch -M main
```

Use that command carefully if the repository already contains branches or history you need to preserve.

5.2 Clone an Existing Repository

If a repository already exists on a remote hosting platform:

```
git clone <repository-url>
```

For example:

```
git clone https://github.com/OWNER/REPOSITORY.git
```

Cloning normally creates a local directory, downloads the repository's history and tracked files, and configures a remote named **`origin`**.

Move into the cloned directory:

```
cd REPOSITORY
```

Check the repository state:

```
git status
git remote -v
```

Which command should you use?

- **`git init`**: Start tracking an existing local directory.
- **`git clone`**: Obtain a local copy of an existing remote repository.

Do not blindly initialize and connect an existing project without checking whether it already has Git history.

---

6. Your First Complete Git Workflow

Suppose you're creating a Java backend project and adding a README.

Step 1: Inspect the project

```
git status
```

This helps you understand the current working-tree state before making changes.

Step 2: Inspect changed files

```
git diff
```

This shows unstaged changes to tracked files.

For a new untracked file, `git diff` will not normally show its contents. Stage it first if you want to inspect the staged diff.

Step 3: Stage the intended changes

```
git add README.md
```

Stage only the file you intend to include.

You can stage multiple specific files:

```
git add README.md pom.xml
```

To stage all changes under the current directory, including deletions and modifications:

```
git add -A
```

Use broad staging commands only after inspecting the repository. Otherwise, you may accidentally stage generated files, local configuration, or secrets.

Step 4: Inspect the staged changes

```
git diff --cached
```

This shows the changes currently staged for the next commit.

Step 5: Commit

```
git commit -m "Add project documentation"
```

A commit records the staged changes in local history.

Step 6: Inspect the result

```
git status
git log -1 --oneline
```

The first command reports the working-tree state. The second shows the latest commit in a compact format.

Remember the workflow

```
Inspect → Edit → Review → Stage → Review Staged Changes → Commit
```

The review steps are important. They reduce accidental commits and make your history easier to understand.

---

7. Essential Git Commands

These are the commands you'll use frequently as a backend engineer.

Command	Purpose
`git status`	Inspect working-tree and staging status
`git init`	Initialize a repository
`git clone <url>`	Clone a remote repository
`git add <file>`	Stage a file
`git add -A`	Stage additions, modifications, and deletions
`git commit -m "message"`	Commit staged changes
`git log`	Inspect commit history
`git log --oneline`	View compact commit history
`git diff`	Inspect unstaged changes to tracked files
`git diff --cached`	Inspect staged changes
`git branch`	List local branches
`git switch <branch>`	Switch branches
`git switch -c <branch>`	Create and switch to a branch

Command	Purpose
`git merge <branch>`	Merge the specified branch into the current branch
`git fetch`	Download remote updates without integrating them into the current branch
`git pull`	Fetch and integrate remote changes
`git push`	Push local commits to a remote
`git stash`	Temporarily store changes in progress
`git restore <file>`	Restore file contents from a source, commonly discarding unstaged changes
`git revert <commit>`	Create a new commit that reverses a specified commit's changes
`git reset`	Move a branch reference and, depending on options, modify the index and working tree

Some commands are intentionally destructive in particular modes. We will examine recovery and safe usage in Chunk 2.

---

8. Branching for Backend Development

Imagine you're building a payment API. Your `main` branch contains the current integrated application, and you want to develop a new payment-validation feature.

Create a feature branch:

```
git switch -c feature/payment-validation
```

Make your changes and commit them:

```
git add src/
git commit -m "Add payment validation"
```

Return to `main`:

```
git switch main
```

Merge the feature branch:

```
git merge feature/payment-validation
```

The merge incorporates the feature branch's changes into the current branch, subject to the histories and any conflicts.

Why use branches?

Branches help engineers:

- Isolate features and bug fixes.
- Work on multiple tasks without mixing their changes.

- Prepare changes for review.
- Keep the main development line more controlled.

Important detail

Switching branches can be blocked when local changes would be overwritten. Commit, stash, or otherwise safely handle those changes before switching.

Never assume switching branches automatically saves your uncommitted work.

---

9. Understand fetch, pull, and push

These commands are fundamental to collaborating through a remote repository.

`git fetch`

```
git fetch origin
```

Downloads updates from the remote and updates the corresponding remote-tracking references.

It does not automatically merge those changes into your current branch.

Use it when you want to inspect remote changes before integrating them.

`git pull`

```
git pull
```

Fetches updates and integrates them into the current branch.

The integration behavior depends on your configuration and options. It may use a merge or a rebase.

`git push`

```
git push origin main
```

Pushes local commits from `main` to the remote branch, assuming the remote and branch are configured appropriately and the operation is permitted.

Mental model

- Remote → Local remote-tracking references: `git fetch`
- Remote → Local branch, with integration: `git pull`
- Local commits → Remote: `git push`

Common mistake

Running `git pull` without understanding the current branch, remote configuration, or working-tree state can produce unexpected results.

Inspect first:

```
git status
git branch --show-current
git remote -v
```

Then choose the appropriate operation.

---

10. Merge Conflicts

A merge conflict occurs when Git cannot automatically reconcile changes.

For example, two developers edit the same lines in `application.yml`.

Git may mark the conflicting region like this:

```
<<<<<<< HEAD
server:
  port: 8080
=======
server:
  port: 8081
>>>>>>> feature/config-update
```

The marker meanings are:

- `<<<<<<< HEAD`: The version from the current branch.
- `=======`: Separates the competing versions.
- `>>>>>>> feature/config-update`: The version from the other branch.

These markers are temporary conflict notation, not valid YAML.

How to resolve a conflict

1. Open the conflicted file.
2. Understand what each version is trying to accomplish.
3. Decide which changes should remain.
4. Edit the file into the correct final form.
5. Remove all conflict markers.
6. Run appropriate tests or validation.
7. Stage the resolved file.
8. Complete the merge or other operation in progress.

For example:

```
git status
```

Inspect the reported conflict paths, edit the files, and then stage a resolved file:

```
git add application.yml
```

If a merge is in progress, complete it as directed by Git. For a normal merge that requires a merge commit:

```
git commit
```

Git may open an editor with a default merge message.

If you decide to abandon an in-progress merge, `git merge --abort` may be appropriate. Inspect your working tree first, particularly if you had uncommitted changes before starting.

Interview insight

A conflict is not necessarily a failure of Git. It is a point where Git needs a human to decide how competing changes should be reconciled.

---

11. restore, revert, and reset

These commands are related to undoing work, but they do different things.

11.1 `git restore`

Restore a tracked file's working-tree contents:

```
git restore UserService.java
```

By default, this discards the file's unstaged modifications by restoring from the index.

To unstage a file without discarding its working-tree changes:

```
git restore --staged UserService.java
```

The second command removes the file from the staging area but leaves the working-tree changes in place.

Caution: restoring a file can discard work. Review the diff before using it.

11.2 `git revert`

```
git revert <commit-hash>
```

Creates a new commit that reverses the changes introduced by the specified commit.

This is commonly useful when a problematic change has already been shared with other developers.

It preserves the existing history and records the reversal.

11.3 `git reset`

```
git reset --soft HEAD~1
```

Moves the current branch reference back one commit while keeping the changes from that commit staged.

Other modes have different effects:

- `--soft`: Keeps changes staged.
- `--mixed`: Resets the index while keeping working-tree changes; this is the default mode.
- `--hard`: Resets the index and working tree, potentially discarding uncommitted changes.

Never use `git reset --hard` casually. It can permanently discard work that is not otherwise recoverable.

Which should you choose?

Situation	Typical choice
Discard unstaged edits to a file	`git restore <file>`
Unstage a file but keep its edits	`git restore --staged <file>`
Reverse a commit while preserving shared history	`git revert <commit>`
Undo a recent local commit while keeping its changes staged	`git reset --soft HEAD~1`
Discard local changes intentionally	A destructive operation only after careful verification

More complex recovery scenarios, including reflog and recovering lost commits, belong in Chunk 2.

---

12. Temporarily Store Work with git stash

Suppose you're halfway through implementing a feature, but you need to switch to another branch to fix an urgent bug.

You do not want to create a premature commit.

You can temporarily stash tracked changes:

```
git stash push -m "Work in progress: payment validation"
```

Inspect the stash list:

```
git stash list
```

Reapply the latest stash while keeping it in the stash list:

```
git stash apply
```

Apply the latest stash and remove it from the stash list if successful:

```
git stash pop
```

Stash application can produce conflicts.

Also, untracked files are not included by default. To include them:

```
git stash push -u -m "Include untracked files"
```

When should you stash?

Use it for short-lived context switching, not as a replacement for meaningful commits or a long-term backup.

---

13. .gitignore and Secret Protection

A `.gitignore` file specifies patterns for files that Git should normally ignore when they are untracked.

A Java/Spring Boot project might use rules such as:

```
# Build output
target/
# IDE configuration
.idea/
.vscode/
# Environment files
.env
.env.*
# Logs
*.log
# Operating-system metadata
.DS_Store
```

Adjust these patterns to your project. For example, if you need to commit a safe example environment file, you can deliberately allow a specific template while keeping real secrets excluded.

Why this matters

Backend projects often contain:

- Database credentials.
- API keys.
- JWT signing secrets.
- Cloud access keys.

- Local environment configuration.
- Generated build output.

Do not commit real secrets.

Important limitation

Adding a file to `.gitignore` does not automatically stop tracking a file that Git already tracks.

If a file has already been committed, removing it from tracking requires a separate operation. Even then, previously committed secrets may remain in repository history or other clones.

If a real credential is exposed, revoke or rotate it immediately. Deleting the file from the latest commit is not sufficient protection.

---

14. Pull Requests and Code Review

A pull request is a platform feature, commonly provided by GitHub, that lets developers propose changes for review before those changes are integrated.

A typical workflow looks like this:

```
main
  |
  +---- feature/payment-validation
                |
                | commits
                v
          Push feature branch
                |
                v
          Open pull request
                |
                v
          Review and tests
                |
                v
          Merge approved changes
```

A useful pull request should explain:

- What changed?
- Why was the change needed?
- How was it tested?
- Are there relevant screenshots, logs, or API examples?
- Are there risks or deployment considerations?

For backend work, reviewers may examine:

- API behavior.
- Database queries and migrations.
- Error handling.
- Authentication and authorization.

- Logging and sensitive-data exposure.
- Test coverage.
- Performance implications.
- Compatibility with existing clients.

Interview insight

A pull request is not just a way to merge code. It is a collaboration and quality-control mechanism.

---

15. Common Git Mistakes

Mistake 1: Forgetting to inspect staged changes

- **Problem**: Unrelated or sensitive files are included in a commit.
- **Prevention**:

```
git status
git diff --cached
```

Mistake 2: Assuming a commit uploads code

- **Problem**: Changes exist locally but not on the remote.
- **Correction**: Push when appropriate, then verify the remote branch.

Mistake 3: Committing secrets

- **Problem**: Credentials may be exposed in repository history.
- **Prevention**: Review staged files and maintain a suitable `.gitignore`. Rotate any exposed credentials.

Mistake 4: Using vague commit messages

- **Weak**:

```
git commit -m "changes"
```

- **Better**:

```
git commit -m "Add payment validation"
```

- A useful message describes the intent of the change.

Mistake 5: Using destructive commands without understanding them

- **Problem**: Uncommitted work can be lost.

- **Prevention**: Inspect `git status`, review diffs, and understand the command's effect before executing it.

Mistake 6: Treating Git and GitHub as the same thing

- **Problem**: Confusion about local commits, remote branches, authentication, and pushing.
- **Correction**: Understand which operations are local and which interact with a remote.

Mistake 7: Using `git add .` without reviewing the project

- **Problem**: Generated files, temporary files, or secrets may be staged.
- **Correction**: Inspect the repository and use targeted staging where appropriate.

---

16. Practical Lab — Create a Clean Git Repository

Complete this exercise in a disposable practice directory or a new project directory that does not contain valuable work.

Do not experiment with destructive commands in an important existing repository.

Task A: Initialize

```
mkdir git-practice
cd git-practice
git init
git branch -M main
```

If Git reports that a repository already exists, stop and inspect rather than assuming the directory is empty.

Task B: Configure your identity if needed

```
git config user.name "Your Name"
git config user.email "you@example.com"
```

These repository-local settings are appropriate for the disposable exercise.

Task C: Create a README

```
echo "# Git Practice" > README.md
```

Inspect the status:

```
git status
```

Task D: Stage and review

```
git add README.md
git diff --cached
```

Task E: Commit

```
git commit -m "Add initial README"
```

Task F: Modify and inspect

```
echo "Learning Git fundamentals" >> README.md
git diff
```

Stage and commit the new change:

```
git add README.md
git diff --cached
git commit -m "Document Git learning goals"
```

Task G: Inspect history

```
git log --oneline
```

You should see two commits.

Task H: Create a branch

```
git switch -c feature/add-notes
```

Add a file:

```
echo "Branching practice" > notes.txt
git add notes.txt
git commit -m "Add branching practice notes"
```

Return to main:

```
git switch main
```

Inspect the directory and history. Notice that the branch-specific commit is not part of `main` yet.

This is the core value of branching: you can develop separately and integrate changes deliberately.

Lab completion criteria

You should be able to explain:

- Why the first commit was local.
- What the staging area did.
- Why the second branch has its own commit.
- What would happen if you merged the feature branch into `main`.
- Why you should inspect changes before committing.

---

17. Git Interview Questions and Answers

Q1. What is Git?

Answer: Git is a distributed version control system that tracks changes to files and allows developers to maintain history, create branches, and collaborate.

Q2. What is the difference between Git and GitHub?

Answer: Git is the version-control system. GitHub is an online platform that hosts Git repositories and adds collaboration features such as pull requests and code reviews.

Q3. What is the staging area?

Answer: It is the intermediate area where you select changes to include in the next commit.

Q4. What is the difference between `git add` and `git commit`?

Answer: `git add` stages changes. `git commit` records the staged changes in the local repository.

Q5. What is a branch?

Answer: A branch is a movable reference to a commit, allowing a line of development to progress independently.

Q6. What is HEAD?

Answer: HEAD usually identifies the currently checked-out commit or branch. In a normal attached state, HEAD points to the current branch, which in turn identifies a commit.

Q7. What is the difference between `git fetch` and `git pull`?

Answer: `git fetch` downloads remote updates without integrating them into the current branch. `git pull` fetches updates and then integrates them according to the configured or selected strategy.

Q8. What is the difference between `git merge` and `git rebase`?

Answer: Merge integrates histories and may create a merge commit. Rebase replays commits onto another base, creating new commit identities for the replayed commits. Rebase can produce a more linear history, but rewriting published history can disrupt collaboration.

Q9. What is a merge conflict?

Answer: It occurs when Git cannot automatically reconcile competing changes. A developer must inspect the conflict, decide the correct result, and complete the operation.

Q10. What is the difference between `git revert` and `git reset`?

Answer: `git revert` creates a new commit that reverses another commit's changes. `git reset` moves a branch reference and can also alter the index and working tree, depending on its mode.

Q11. What is `.gitignore`?

Answer: It specifies patterns for files Git should normally ignore when they are untracked. It does not automatically untrack files already committed.

Q12. Why are small, meaningful commits useful?

Answer: They make changes easier to review, debug, revert, and understand.

Q13. What does `git stash` do?

Answer: It temporarily stores changes so you can switch context without immediately committing those changes.

Q14. Does `git commit` push code to GitHub?

Answer: No. A commit records changes in the local repository. Pushing transfers commits to a remote repository.

Q15. What should you check before committing?

Answer: Inspect the working-tree status, review the intended changes, check staged files with `git diff --cached`, and confirm that no secrets or unrelated files are included.

---

18. Mastery Checklist

Before proceeding to Chunk 2, verify that you can:

- [ ] Explain Git versus GitHub.
- [ ] Explain the working directory, staging area, and local repository.
- [ ] Initialize or clone a repository appropriately.
- [ ] Configure Git identity.
- [ ] Inspect repository status.

- [ ] Stage selected files.
- [ ] Review staged and unstaged changes.
- [ ] Create meaningful commits.
- [ ] Inspect commit history.
- [ ] Create and switch branches.
- [ ] Explain fetch, pull, and push.
- [ ] Merge a feature branch.
- [ ] Explain how merge conflicts are resolved.
- [ ] Distinguish restore, revert, and reset.
- [ ] Stash and reapply changes.
- [ ] Configure a useful `.gitignore`.
- [ ] Explain why `.gitignore` does not remove previously committed secrets.
- [ ] Describe a pull request and its purpose.

---

19. What Comes Next?

Git Chunk 2 of 2 — Collaboration, Recovery, Advanced Workflows & Interview Scenarios

The next chunk will build upon these foundations and cover:

- Remote branches and upstream tracking.
- More deliberate collaboration workflows.
- Cherry-picking commits.
- Reflog and recovery techniques.
- Diagnosing and recovering from common Git mistakes.
- Safe history management.
- Practical scenarios involving feature branches and shared repositories.
- Advanced Git interview questions.
- A final Git mastery assessment.

Once both chunks are complete, we will assemble and verify the single complete Git handbook PDF:

`BEH_GIT_COMPLETE_HANDBOOK.pdf`

Only after that PDF is verified will we create the Git completion commit:

```
Git done
```

No separate chunk PDFs will be created.

PART II: COLLABORATION, RECOVERY & PRODUCTION ENGINEERING

Phase I — Developer Foundations | Chunk 2 of 2

Git Chunk 2 of 2

Collaboration, Recovery & Production Engineering

Backend Engineering Tools & Infrastructure Handbook

Phase I — Developer Foundations

1. The Mission: From Using Git to Engineering with Git

In Chunk 1, we established the fundamentals:

- The working directory, staging area, and repository.
- Commits, branches, and HEAD.
- Essential Git commands.
- Basic branching and merging.
- Conflict resolution.
- `restore`, `revert`, and `reset`.
- Stashing and `.gitignore`.
- Pull requests and everyday development workflows.

Now we take the next step.

Imagine you're working on a production Spring Boot application with five other engineers.

One developer introduces a regression. Another pushes a conflicting change. You accidentally commit to the wrong branch. A deployment breaks, and the team needs to identify the responsible commit quickly.

Knowing `git add` and `git commit` isn't enough.

You must understand how Git behaves when multiple histories interact, how to preserve work during recovery, and how to make safe decisions when a repository contains important shared history.

This chunk develops three complementary layers:

Layer	Objective
Layer 1 — Fundamentals & Mental Models	Understand collaboration, history, references, and recovery
Layer 2 — Practical Engineering & Workflows	Execute branching, integration, investigation, and recovery workflows

Layer	Objective
Layer 3 — Production Engineering & Interview Mastery	Handle regressions, shared-history risks, incident investigations, and advanced interview scenarios

By the end, you should be able to use Git as both a development tool and an engineering investigation tool.

LAYER 1 — FUNDAMENTALS & MENTAL MODELS

2. Git Collaboration: Understand the Repository You're Working In

2.1 Local branches and remote-tracking branches

Suppose your team hosts a repository on GitHub.

You clone it and begin developing locally.

You may see these references:

- `main`
- `origin/main`
- `feature/payment-validation`
- `origin/feature/payment-validation`

They represent different things:

Reference	Meaning
`main`	Your local branch
`origin/main`	Your local record of the remote's main branch
`feature/payment-validation`	Your local feature branch
`origin/feature/payment-validation`	Your local record of the remote feature branch

The distinction is important: `origin/main` is not a live connection to GitHub. It is a remote-tracking reference updated by operations such as `git fetch`.

The remote named `origin` is simply a conventional name. A repository can have other remotes.

2.2 Inspect before changing anything

Use:

```
git status
git branch -vv
git branch -a
git remote -v
git log --oneline --graph --decorate --all
```

These commands answer important questions:

- What branch am I on?
- Do I have uncommitted work?
- Which remote branches are known locally?
- Which remote does this repository use?
- How are the commits connected?

Engineering principle: Never start a complex Git recovery or synchronization operation without first understanding the repository's current state.

2.3 What is an upstream branch?

An upstream branch is the branch associated with your local branch for comparison and synchronization.

For example:

```
Local branch: feature/payment-validation
Upstream:     origin/feature/payment-validation
```

Inspect upstream relationships:

```
git branch -vv
```

When a local branch has an upstream configured, Git can generally infer the appropriate remote branch for ordinary `git pull` and `git push` commands.

The upstream relationship does not mean that your local and remote branches are always synchronized.

They can diverge.

---

3. Understand History Before Manipulating It

A Git commit is a snapshot of tracked project state, together with metadata and references to parent commits.

A branch is a reference to a commit.

`HEAD` identifies the currently checked-out commit or branch, depending on the repository state.

These concepts explain why many Git operations behave as they do.

Consider:

```
A --- B --- C       main
       \
        D --- E     feature
```

Here:

- `main` points to commit `C`.
- `feature` points to commit `E`.
- Both branches share history through commit `B`.
- Commits `D` and `E` are not part of the ancestry of `main`.

If you merge `feature` into `main`, Git integrates the feature history into `main`.

If you rebase `feature` onto `main`, Git replays the feature's commits on top of `main`, producing new commit identities.

If you cherry-pick `D` onto `main`, Git applies the changes introduced by `D` and creates a new commit on `main`.

These operations can produce similar file contents while creating different histories.

The critical distinction: file contents and commit history are related, but they are not the same thing.

---

4. Merge, Rebase, and Cherry-Pick: Three Different Decisions

4.1 Merge: Integrate histories

Suppose you have:

```
A --- B --- C       main
       \
        D --- E     feature
```

You run:

```
git switch main
git merge feature
```

If a fast-forward is not possible and Git creates a merge commit, the history may become:

```
A --- B --- C ------- M    main
       \             /
        D --- E -----
```

Merge integrates the histories while preserving the original commits.

Advantages:

- Preserves branch topology.
- Clearly records the integration point when a merge commit is created.
- Works well for integrating shared development histories.

Trade-off:

- Merge commits can make the history graph more complex.

A merge does not always create a merge commit. If the current branch can fast-forward to the other branch, Git may simply move the branch reference forward.

4.2 Rebase: Replay commits onto another base

Run:

```
git switch feature
git rebase main
```

Conceptually, the history becomes:

```
A --- B --- C             main
            \
             D' --- E'    feature
```

Git replays the feature commits on top of `main`.

The replayed commits have new identities because their parent relationships have changed.

Advantages:

- Can produce a more linear history.
- Makes some commit sequences easier to inspect.

Trade-offs:

- Rewrites the identities of replayed commits.
- May require resolving conflicts during replay.
- Can disrupt collaboration if published history is rewritten carelessly.

Rebase is often appropriate for unpublished feature work.

Avoid rewriting shared branch history unless the team explicitly agrees on the workflow.

4.3 Cherry-pick: Apply one specific commit

Suppose commit `D` contains a bug fix, but commit `E` contains unrelated work.

You want only the fix:

```
git switch main
git cherry-pick <commit-hash-of-D>
```

Git applies the changes introduced by `D` to the current branch and creates a new commit.

The original commit remains in its original history.

Cherry-picking is useful for:

- Backporting a fix to a maintenance branch.
- Applying a targeted correction to a release branch.
- Transferring an isolated change without merging an entire feature branch.

Avoid indiscriminate cherry-picking. Repeatedly copying commits across branches can complicate future integration.

4.4 Decision table

Situation	Typical approach
Integrate a feature branch	Merge or the team's approved pull-request strategy
Replay private feature commits onto an updated base	Rebase
Apply one isolated fix to another branch	Cherry-pick
Reverse a published problematic commit	Revert
Remove or reorganize unpublished local commits	Reset or rebase, with care

No operation is universally best. The correct choice depends on the intended history, collaboration policy, and whether commits have been shared.

LAYER 2 — PRACTICAL ENGINEERING & WORKFLOWS

5. The Professional Feature-Branch Workflow

Imagine you're implementing payment validation in a Spring Boot application.

The team expects features to be developed on separate branches and reviewed before integration.

Step 1: Inspect your current state

```
git status
git branch --show-current
git branch -vv
```

Do not switch branches or pull remote changes blindly when you have uncommitted work.

Step 2: Update your local main branch

```
git switch main
git fetch origin
git pull --ff-only origin main
```

The fast-forward-only option prevents Git from automatically merging or rebasing a diverged local branch during this pull.

If the operation fails because the histories have diverged, inspect them and decide on an appropriate integration strategy.

Step 3: Create a feature branch

```
git switch -c feature/payment-validation
```

Step 4: Implement and commit the feature

```
git add src/
git diff --cached
git commit -m "Add payment validation"
```

Inspect the staged diff before committing. Ensure that only intended files are included.

Step 5: Push and establish upstream tracking

```
git push -u origin feature/payment-validation
```

The `-u` option configures the upstream relationship if the push succeeds.

Step 6: Open a pull request

Propose merging the feature branch into the appropriate target branch.

A strong pull request should describe:

- What changed.
- Why it changed.
- How it was tested.
- Whether database migrations are required.
- Whether configuration changes are required.
- Any deployment risks.
- Any compatibility implications.

Step 7: Integrate the approved change

The team may use a merge commit, squash merge, or rebase-based strategy.

Follow the repository's conventions rather than assuming all teams use the same strategy.

Step 8: Synchronize your local main branch

After the feature has been integrated:

```
git switch main
git fetch origin
git pull --ff-only origin main
```

Delete obsolete branches only after confirming that their work has been preserved and integrated.

Delete a local branch:

```
git branch -d feature/payment-validation
```

Delete a remote branch only when appropriate:

```
git push origin --delete feature/payment-validation
```

Mastery principle: A branch is a way to organize development, not a backup. Preserve important commits and verify integration before deleting branches.

---

6. Scenario: Your Push Is Rejected

You run:

```
git push origin main
```

Git responds with a non-fast-forward rejection.

This often means the remote branch contains commits that your local branch does not have.

For example:

```
Local main:   A --- B --- C
Remote main:  A --- B --- D
```

Both histories share `A` and `B`, but they have diverged.

Do not immediately force-push.

Step 1: Inspect the repository

```
git status
git branch -vv
```

Step 2: Fetch remote changes

```
git fetch origin
```

Step 3: Compare the histories

```
git log --oneline --graph --decorate main origin/main
```

See commits on the remote that are not on your local branch:

```
git log main..origin/main --oneline
```

See commits on your local branch that are not on the remote:

```
git log origin/main..main --oneline
```

These comparisons assume that both references exist locally.

Step 4: Choose an integration strategy

If your local branch can safely fast-forward, you can use:

```
git pull --ff-only origin main
```

If the branches have diverged, you must decide whether to merge, rebase unpublished local commits, or use another repository-approved approach.

Before changing history, establish whether the local commits have been shared or whether other developers depend on them.

Step 5: Push after integration

Once the histories are correctly integrated and the changes are tested:

```
git push origin main
```

Why not immediately force-push?

A force-push can replace the remote branch reference with a different history, potentially making other developers' commits inaccessible through that branch.

`--force-with-lease` offers a safety check against unexpected remote changes, but it does not eliminate the risk of rewriting shared history.

Rule: A rejected push is a signal to inspect the histories, not an invitation to overwrite the remote.

---

7. Scenario: You Committed to the Wrong Branch

You intended to commit a feature on `feature/authentication`, but you committed it to `main`.

Your latest commit contains the feature.

The correct recovery depends on whether the commit has been pushed.

Case A: The commit is local and unpublished

First, inspect:

```
git status
git log --oneline -5
```

If the latest commit is the mistaken feature commit and the working tree is clean, create the intended branch at the current commit:

```
git branch feature/authentication
```

This preserves the commit under the new branch name.

If you have confirmed that the latest commit should be removed from the local `main` branch, you can move `main` back one commit:

```
git switch main
git reset --hard HEAD~1
```

Warning: This is destructive to uncommitted changes and moves the current branch reference. Use it only after verifying the commit, confirming the feature branch preserves the desired work, and ensuring no uncommitted changes need protection.

Then switch to the feature branch:

```
git switch feature/authentication
```

Verify the branch and history:

```
git log --oneline --graph --decorate --all
```

Case B: The commit has already been pushed

Do not automatically reset the shared branch and force-push.

A safer general approach is to reverse the unwanted change on the shared branch:

```
git revert <commit-hash>
```

Then ensure the feature is available on the intended branch and follow the team's normal review and integration workflow.

The exact sequence depends on the branch topology and whether the feature has already been copied or integrated elsewhere.

The lesson

Preserve the desired commit before changing a branch reference. Treat unpublished local history differently from shared history.

---

8. Scenario: A Commit Introduced a Regression

Your backend API worked yesterday. Today, a validation endpoint returns HTTP 500 instead of the expected response.

You suspect a recent code change.

Git provides several ways to investigate:

8.1 Inspect recent history

```
git log --oneline -10
```

8.2 Inspect a suspicious commit

```
git show <commit-hash>
```

8.3 Compare two revisions

```
git diff <known-good-commit> <failing-commit>
```

To focus on one file:

```
git diff <known-good-commit> <failing-commit> -- src/main/java/com/example/UserService.java
```

8.4 Inspect a file's history

```
git log --oneline -- src/main/java/com/example/UserService.java
```

These commands help narrow down what changed, but they do not prove that a particular commit caused the failure.

You must also reproduce the failure and examine the runtime evidence.

---

9. Find the Problematic Commit with git bisect

When many commits exist between the last known-good revision and the failing revision, `git bisect` can systematically narrow down the regression.

Suppose:

- Commit `A` passes the test.
- Commit `H` fails the test.
- Several commits exist between them.

Git can repeatedly select an intermediate revision for testing.

Step 1: Start bisect

```
git bisect start
```

Step 2: Mark the current revision as bad

```
git bisect bad
```

Step 3: Mark a known-good revision

```
git bisect good <known-good-commit>
```

Git checks out a revision between the good and bad points.

Run a reliable regression test.

If it fails:

```
git bisect bad
```

If it passes:

```
git bisect good
```

Repeat until Git identifies the first bad commit in the tested range, assuming the good/bad classification is consistent and the history supports the search.

Step 4: End the session

```
git bisect reset
```

This returns you to the previous checkout state.

Backend example

Suppose you have a repeatable integration test that submits an invalid request and checks the response.

If the test passes on the known-good revision but fails on the current revision, you can use it to classify revisions during bisect.

Important limitations

Bisect is most useful when:

- The known-good and known-bad revisions are identified correctly.
- The test is repeatable.
- The environment is consistent.
- The relevant commits form a suitable history for the search.

Flaky tests, build failures unrelated to the regression, and inconsistent dependencies can make the results difficult to interpret.

---

10. Recovery: You Cannot Find a Commit

You reset a branch and a commit disappeared from normal history.

Before assuming the work is lost, inspect the reflog:

```
git reflog
```

You might see:

```
a1b2c3d HEAD@{0}: reset: moving to HEAD~1
d4e5f6a HEAD@{1}: commit: Add payment validation
```

The hashes and descriptions are illustrative.

The older entry indicates a previous **`HEAD`** position.

Step 1: Inspect the candidate commit

```
git show d4e5f6a
```

Step 2: Preserve it with a recovery branch

```
git branch recovery/payment-validation d4e5f6a
```

Step 3: Inspect the recovered history

```
git log --oneline recovery/payment-validation
```

Once you have confirmed the recovered commit, determine how it should be integrated.

Why not immediately reset the current branch again?

You may have identified the wrong commit or misunderstood the repository state.

A recovery branch preserves the candidate while you investigate.

Reflog limitations

Reflog is a local recovery aid, not a permanent backup.

It does not guarantee recovery of uncommitted work or every unreachable object. Entries can expire, and unreachable objects can eventually be removed by repository maintenance.

If work appears lost, stop running cleanup commands and investigate before making further changes.

---

11. Recovery: Choosing Between Reset and Revert

Suppose commit `D` introduces a bug:

```
A --- B --- C --- D
```

The correct recovery strategy depends on whether the commit is private or shared.

11.1 Revert a shared commit

```
git revert <commit-hash>
```

Git creates a new commit that reverses the selected commit's changes.

Conceptually:

```
A --- B --- C --- D --- E
                        ^
                  Reversal of D
```

The original history remains visible.

This is generally suitable for correcting published changes without rewriting the shared branch history.

11.2 Reset an unpublished commit

If the latest commit is local, unpublished, and you want to keep its changes staged:

```
git reset --soft HEAD~1
```

If you want to keep its changes in the working tree but unstage them:

```
git reset HEAD~1
```

The default reset mode is `--mixed`.

To remove the latest commit and discard changes in the index and working tree:

```
git reset --hard HEAD~1
```

Do not run the hard reset unless you have verified the exact target and protected any work you need.

Decision table

Situation	Typical approach
Latest unpublished commit needs editing	`git reset --soft HEAD~1`
Latest unpublished commit should be undone but its changes retained unstaged	`git reset HEAD~1`
A published commit must be reversed	`git revert <commit>`
A commit appears to have disappeared	Inspect `git reflog`
A particular commit is needed on another branch	`git cherry-pick <commit>`

LAYER 3 — PRODUCTION ENGINEERING & INTERVIEW MASTERY

12. Production Incident: A Bad Configuration Reaches Production

Your Spring Boot application deploys a new release.

Immediately afterward:

- Database connections fail.
- Some API endpoints return HTTP 500.
- The deployment health check fails.
- The team suspects a recent configuration change.

Git can help identify the change, but it cannot independently establish the root cause.

Use a disciplined investigation:

Step 1: Identify the deployed revision

Determine which commit was deployed using release metadata, build information, deployment logs, or the platform's release history.

Do not assume that the latest commit on `main` is the deployed commit.

Step 2: Inspect the relevant change

```
git show <deployed-commit>
```

Compare the deployed revision with the last known-good revision:

```
git diff <known-good-commit> <deployed-commit> -- application.yml
```

Adapt the path to the actual configuration file.

Step 3: Gather runtime evidence

Inspect:

- Application logs.
- Deployment logs.
- Database connectivity.
- Environment-variable configuration.
- Health-check responses.
- Recent infrastructure changes.

Avoid printing secrets into logs or incident reports.

Step 4: Decide whether the commit is responsible

Look for evidence connecting the change to the failure.

A recent commit is a candidate, not proof of causation.

Step 5: Restore service

Depending on the deployment system and incident, you may:

- Revert the offending commit and deploy the correction.
- Apply a forward fix.
- Roll back to a known-good release.

Follow the team’s release and incident procedures.

Step 6: Verify recovery

Confirm that:

- The application starts correctly.
- Database connections succeed.
- Health checks pass.
- The affected API behavior is restored.
- Relevant error rates return to acceptable levels.

Step 7: Preserve the learning

Create a focused fix, add regression tests, and document the incident according to team policy.

Production takeaway

Git provides the history of intended code changes. Logs, metrics, deployment metadata, and runtime tests provide the evidence needed to understand actual system behavior.

---

13. Git Safety Rules for Production Teams

Rule 1: Inspect before changing history

Before `reset`, `rebase`, `cherry-pick`, or recovery operations:

```
git status
git log --oneline --graph --decorate --all
git branch -vv
```

These commands do not guarantee safety, but they give you the context needed to make a reasoned decision.

Rule 2: Preserve valuable commits before recovery

If you are uncertain whether a commit is needed, create a branch at that commit after verifying it:

```
git branch recovery/important-work <commit-hash>
```

A branch reference can preserve access to a commit while you investigate.

Rule 3: Treat shared history carefully

Rewriting unpublished local history is different from rewriting a branch used by other developers. Coordinate before rewriting shared history.

Rule 4: Do not treat Git as a backup system

A repository can be damaged, local commits can become unreachable, and uncommitted changes may never enter Git history. Use appropriate remote backups and preserve important work.

Rule 5: Protect secrets immediately

If a credential is committed:

1. Revoke or rotate it.
2. Identify potentially exposed copies.
3. Assess whether history cleanup is necessary.

4. Coordinate any history rewrite.
5. Prevent the secret from being committed again.

Removing the secret from the latest commit does not invalidate copies of the old credential.

Rule 6: Keep commits understandable

Prefer focused commits with meaningful messages.

Examples:

- `Add payment validation`
- `Fix database connection configuration`
- `Handle duplicate event delivery`
- `Add retry handling to payment consumer`

Good commit messages help code review, regression investigation, and incident analysis.

---

14. Practical Engineering Lab

Complete the following exercises in a disposable repository.

Do not perform destructive experiments in a project containing valuable work.

Lab A: Create and push a feature branch

1. Initialize or clone a practice repository.
2. Create `feature/api-validation`.
3. Make a change.
4. Commit it.
5. Push with upstream tracking.
6. Inspect the local and remote-tracking references.

Commands to practise:

```
git switch -c feature/api-validation
git add .
git diff --cached
git commit -m "Add API validation"
git push -u origin feature/api-validation
git branch -vv
```

Lab B: Cherry-pick an isolated fix

1. Create a feature branch.
2. Add and commit a single isolated change.
3. Switch to another branch.
4. Cherry-pick the commit.
5. Compare the resulting histories.

Explain why the cherry-picked commit has a different identity.

Lab C: Recover a commit

1. Create a commit in a disposable repository.
2. Record its hash.
3. Move the branch reference backward.
4. Inspect `git reflog`.
5. Create a recovery branch at the intended commit.
6. Verify that the expected content is present.

Do not proceed with further cleanup until the recovery has been verified.

Lab D: Investigate a regression

1. Create several commits.
2. Introduce a deliberate bug in one commit.
3. Add a repeatable test that exposes it.
4. Identify known-good and known-bad revisions.
5. Use `git bisect` to locate the problematic commit.
6. End the session with `git bisect reset`.

Lab E: Handle a rejected push

Using a remote practice repository and two separate working copies:

1. Commit a change in the first copy.
2. Independently commit another change in the second copy.
3. Push from one copy.
4. Attempt to push the other copy.
5. Inspect the rejection.
6. Fetch and compare histories.
7. Integrate the remote changes using an appropriate strategy.
8. Verify the final history and push successfully.

This exercise demonstrates why a rejected push should trigger investigation rather than an immediate force-push.

---

15. Advanced Interview Questions

Q1. What is the difference between `origin/main` and `main`?

Answer: `main` is a local branch. `origin/main` is a remote-tracking reference representing the local repository's view of the remote branch.

Q2. Why does cherry-picking create a new commit?

Answer: Cherry-pick applies the changes from an existing commit to the current branch. The resulting commit has a different parent relationship and therefore a different commit identity.

Q3. How can you recover a commit after an accidental reset?

Answer: Inspect `git reflog`, identify the intended commit, verify it with `git show`, and preserve it using a recovery branch before deciding how to integrate it.

Q4. Why is reflog not a backup?

Answer: It is local, time-limited by repository configuration and maintenance, and cannot guarantee recovery of uncommitted changes or every unreachable object.

Q5. When would you choose revert over reset?

Answer: Revert is generally preferred for reversing changes in shared history because it adds a new commit instead of moving the shared branch backward.

Q6. How do you diagnose a non-fast-forward push rejection?

Answer: Inspect the working tree and branch relationships, fetch the remote, compare local and remote histories, integrate the changes using an appropriate strategy, test, and push again.

Q7. What is the danger of rebasing a shared branch?

Answer: Rebase creates new commit identities. Rewriting published history can cause divergence and force collaborators to reconcile their local histories.

Q8. What is `git bisect`?

Answer: A regression-isolation tool that uses known-good and known-bad revisions to narrow down the commit that introduced a problem.

Q9. How can Git help with a production incident?

Answer: It identifies the code history associated with a deployed revision, enables comparisons with known-good versions, and helps locate candidate changes. Runtime evidence is still necessary to establish causation.

Q10. Why is a clean working tree important before recovery?

Answer: Uncommitted changes can be overwritten or complicated by operations that switch branches, reset files, or rewrite history. A clean tree makes the starting state easier to reason about, although it does not by itself guarantee safety.

Q11. How should a team handle a commit that accidentally reached a shared production branch?

Answer: Assess the impact, follow the team's incident process, and usually create a corrective commit or revert the change. Avoid rewriting shared history without explicit coordination.

Q12. When should you use cherry-pick instead of merge?

Answer: When you need a specific isolated change from another branch and do not want to integrate the entire branch history.

Q13. Why is `git reset --hard` dangerous?

Answer: It moves the branch reference and resets the index and working tree, potentially discarding uncommitted changes. Its use requires careful verification.

Q14. What is the difference between a code rollback and a Git revert?

Answer: A Git revert creates a commit that reverses changes in repository history. A deployment rollback restores a previously released version of a running system. They are related but not identical operations.

Q15. Can the latest commit always be assumed to be the version running in production?

Answer: No. The deployed revision must be established using release metadata, build identifiers, or deployment records.

---

16. Git Mastery Assessment

Before considering Git complete, you should be able to demonstrate all three layers:

Layer 1 — Fundamentals & Mental Models

- [ ] Explain local branches versus remote-tracking branches.
- [ ] Explain upstream tracking.
- [ ] Describe the differences between merge, rebase, and cherry-pick.
- [ ] Explain why rebased and cherry-picked commits have different identities.
- [ ] Explain reflog and its limitations.

Layer 2 — Practical Engineering & Workflows

- [ ] Execute a feature-branch workflow.
- [ ] Push a branch with upstream tracking.
- [ ] Investigate a rejected push.
- [ ] Cherry-pick an isolated commit.
- [ ] Recover a commit with reflog.
- [ ] Use `git bisect` to locate a regression.
- [ ] Resolve conflicts during integration.
- [ ] Inspect branch differences and commit history.

Layer 3 — Production Engineering & Interview Mastery

- [ ] Distinguish local recovery from shared-history recovery.
- [ ] Explain when revert is safer than reset.
- [ ] Investigate a production regression using deployed revision information.
- [ ] Explain why force-pushing shared branches is risky.
- [ ] Explain why Git is not a backup system.
- [ ] Describe how to handle a committed secret.
- [ ] Distinguish a Git revert from a deployment rollback.
- [ ] Explain how code history and runtime evidence work together during an incident.

---

17. Git Handbook Completion Protocol

Git Chunk 2 of 2 is complete.

The Git curriculum remains unchanged: two chunks total.

The authoritative source files should be:

```
content/git/
■■■ 01-git-fundamentals.md
■■■ 02-git-collaboration-recovery-advanced-workflows.md
```

The completed Git handbook must combine both chunks in order and retain the full educational content.

Final PDF:

```
BEH_GIT_COMPLETE_HANDBOOK.pdf
```

Before marking Git complete:

1. Verify both source files are present and complete.
2. Confirm all three learning layers are represented.
3. Assemble the complete handbook without summarizing or deleting sections.
4. Generate and inspect the PDF.
5. Check code examples, headings, page breaks, and missing content.
6. Update the README only after verification.
7. Commit the completed Git source, PDF, and relevant progress updates using:

```
   Git done
```

8. Verify the commit.
9. Push only if the remote is correctly configured and the push succeeds.

No separate chunk PDFs will be created.

Next in the Curriculum

Linux Chunk 1 of 6 — Terminal & Filesystem Mastery

The total curriculum remains locked at 32 chunks across 13 tools. We are strengthening the structure and quality of the existing chunks, not increasing the number of chunks.